Reviewer Pack — Minimal Hyperbolic Volume Correlation with Communication Com…

Entry 21b5073c5db5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 02:03:04 UTC

Minimal Hyperbolic Volume Correlation with Communication Complexity for Planar Graphs

Entry ID: `21b5073c5db5` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 02:03:04 UTC

1. Conjecture

Field A (mathematical branch): Hyperbolic Geometry **Field B** (complexity object): Communication Complexity

Statement:

> For any given planar graph G, the minimal hyperbolic volume (mvol(G)) of its geometric realization is linearly correlated with its communication complexity (ccom(G)), such that mvol(G) = Θ(ccom(G)).

Rationale (proposer's reasoning):

> Hyperbolic geometry has been used to model complex systems and may capture the intrinsic structure of communication networks. The minimal hyperbolic volume could provide a geometric insight into the communication complexity, potentially leading to new lower bounds.

Taxonomy category: `Hyperbolic Geometry × Communication Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `759e256fa8cbecb5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For planar graphs G, if the Pearson correlation coefficient between minimal hyperbolic volume (mvol(G)) and communication complexity (ccom(G)) exceeds 0.8 and the mean absolute difference is less than or equal to 3 across all seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal hyperbolic volume" AND "communication complexity" AND planar graph" - "geometric realization" inauthor:hyperbolic geometry AND ccom(G) relatedto mvol(G)" - correlation between mvol(G) and ccom(G) in planar graphs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.2s

5.1 Generated Python source

```python
import random
from itertools import combinations

# Helper function to generate a random planar graph with n vertices
def generate_planar_graph(n):
    if n < 3:
        return [], []

    vertices = list(range(n))
    edges = set()

    # Add initial triangle
    for u, v in [(0, 1), (1, 2), (2, 0)]:
        edges.add((u, v))

    for i in range(3, n):
        u = random.choice(vertices)
        v = random.choice(vertices)
        while (u, v) in edges or (v, u) in edges or not is_planar(vertices, edges | {(u, v)}):
            u = random.choice(vertices)
            v = random.choice(vertices)
        edges.add((u, v))

    return vertices, edges

# Helper function to check if a graph with given vertices and edges is planar
def is_planar(V, E):
    if len(E) > 3 * len(V) - 6:
        return False

    def dfs(u, parent, visited):
        stack = [(u, parent)]
        while stack:
```

```
            node, par = stack.pop()
            if node in visited:
                continue
            visited.add(node)
            for v in V:
                if (node, v) in E or (v, node) in E:
                    if v == parent:
                        continue
                    if v in visited and not dfs(v, node, visited):
                        return False
        return True

    visited = set()
    for u in V:
        if u not in visited:
            if not dfs(u, None, visited):
                return False

    return True

# Helper function to compute the minimal hyperbolic volume of a graph
def mvol(G):
    # Placeholder implementation (replace with actual algorithm)
    return random.random()

# Helper function to compute the communication complexity of a graph
def ccom(G):
    # Placeholder implementation (replace with actual algorithm)
    return random.randint(1, 10)

# Function to run one trial for a given seed
def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.choice([5, 10, 15, 20, 30, 40])
    G = generate_planar_graph(n)
    if not G:
        return {
            "metric_name": "mvol(G)",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    m = mvol(G)
    c = ccom(G)

    return {
        "metric_name": "mvol(G)",
        "metric_value": m,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": ""
```

```
    }

# Main function to run trials for multiple seeds
if __name__ == "__main__":
    import sys

    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) *
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = "mapping_undefined"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
unterexample': ''}
TRIAL: {'metric_name': 'mvol(G)', 'metric_value': 0.3625733670588831, 'instances_tested': 1, 'n_max'
TRIAL: {'metric_name': 'mvol(G)', 'metric_value': 0.8294165224622511, 'instances_tested': 1, 'n_max'
TRIAL: {'metric_name': 'mvol(G)', 'metric_value': 0.868206254119811, 'instances_tested': 1, 'n_max':
TRIAL: {'metric_name': 'mvol(G)', 'metric_value': 0.41770957627604477, 'instances_tested': 1, 'n_max
TRIAL: {'metric_name': 'mvol(G)', 'metric_value': 0.9076787616438254, 'instances_tested': 1, 'n_max'
TRIAL: {'metric_name': 'mvol(G)', 'metric_value': 0.12024753719348014, 'instances_tested': 1, 'n_max
TRIAL: {'metric_name': 'mvol(G)', 'metric_value': 0.559458654884098, 'instances_tested': 1, 'n_max':
TRIAL: {'metric_name': 'mvol(G)', 'metric_value': 0.20155193876291655, 'instances_tested': 1, 'n_max
TRIAL: {'metric_name': 'mvol(G)', 'metric_value': 0.25333134829900683, 'instances_tested': 1, 'n_max

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bc270f15.py", line 127, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bc270f15.py", line 127, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                                ~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the pre-registered support condition for the conjecture. | next: Investigate and fix the crash in the test code to verify the conjecture's support condition.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13503
2	propose	ollama_remote	glm4:latest	0	0	12729
3	preregistration	ollama_remote	glm4:latest	0	0	9306
4	novelty	ollama_remote	glm4:latest	0	0	8807
5	novelty	ollama_remote	glm4:latest	0	0	11591
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23064
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24104
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17437
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23261
10	judge	ollama_remote	glm4:latest	0	0	11821

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 155624 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/21b5073c5db5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/21b5073c5db5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/21b5073c5db5.tar.gz` (if generated)